Neapolitan Artistic Heritage Dialogue System

A speech-enabled chatbot for Caravaggio and Battistello Caracciolo's works in Naples, grounded in a DBpedia-derived knowledge graph with inconsistency detection

---

Course: Natural Language Processing (U3539) — Prof. Francesco Cutugno

Student: Mahdi Miri (D03000089) — M.Sc. Data Science

Institution: University of Naples Federico II, DIETI

1. Introduction

This report documents what I built to satisfy my Natural Language Processing course requirement: a speech chatbot, with voice input and text-to-speech output, that answers questions about the works of Caravaggio and Battistello Caracciolo that can be visited in Naples. The brief was specific: the factual knowledge behind the chatbot has to come from SPARQL queries against DBpedia, and the resulting data has to be loaded into a Neo4j graph database. On top of that, the system has to detect inconsistencies in what the user says, and keep track of the conversation across multiple turns rather than treating every question in isolation.

I kept the scope deliberately narrow: two painters, eight paintings, five venues, all within the city of Naples. That narrowness was the point. A general-purpose language model answering questions about art history will happily make things up — assign a painting to the wrong artist, claim a Roman painting is on display in Naples, or invent a museum address. For a cultural-heritage guide that visitors might actually rely on, I consider that kind of fabrication a real problem. Rather than fighting hallucination after the fact, I tried to avoid it by construction: every factual claim the chatbot makes is looked up in a graph I built from verified DBpedia data, and every claim the user makes is checked against that same graph before the system responds.

In the rest of this report I walk through what I built: the data pipeline, the graph database, the dialogue manager, the inconsistency detector, the speech components, and finally my evaluation results, including an honest look at where the approach falls short.

2. Domain: Caravaggio and Battistello in Naples

Caravaggio spent two periods in Naples (1606–1607 and 1609–1610), fleeing Rome after a fatal duel, and his time there left a lasting mark on local painting. Battistello Caracciolo (1578–1635) was the most prominent Neapolitan painter to pick up Caravaggio's style of harsh lighting and unidealized figures, and he was the obvious second subject for a project focused on this period. The catalog I put together covers three Caravaggio works and five Battistello works, all physically on display in Naples today:

Title	Artist	Year	Venue
The Seven Works of Mercy	Caravaggio	1607	Pio Monte della Misericordia
The Flagellation of Christ	Caravaggio	1607	Museo Nazionale di Capodimonte
The Martyrdom of Saint Ursula	Caravaggio	1610	Gallerie d'Italia, Palazzo Zevallos

Title	Artist	Year	Venue
The Liberation of Saint Peter	Battistello Caracciolo	1615	Pio Monte della Misericordia
The Washing of the Feet	Battistello Caracciolo	1622	Certosa di San Martino
Christ at the Column	Battistello Caracciolo	1625	Museo Nazionale di Capodimonte
Trinitas Terrestris	Battistello Caracciolo	1617	Chiesa della Pietà dei Turchini
Frescoes of the Chapel of the Assunta	Battistello Caracciolo	1631	Certosa di San Martino

Two venues — Pio Monte della Misericordia and Museo Nazionale di Capodimonte — each hold works by both painters, which I found useful for testing: it is exactly the kind of overlap where a careless system might attribute the wrong work to the wrong artist at the right building, or vice versa. To test the out-of-scope and city-mismatch detectors honestly, I also kept a short list of well-known Caravaggio paintings that are *not* in Naples — Judith Beheading Holofernes and The Calling of Saint Matthew in Rome, Bacchus in Florence — so the system has something concrete to correct against when a user assumes they can see those works locally.

3. System Overview

I split the system into four layers, each with a single, fairly narrow job. A data-acquisition layer queries DBpedia for the two artists, their works, and the venues, and caches what it gets back locally. A Neo4j graph store holds that catalog as a chain of relationships — artist painted artwork, artwork located at a venue, venue in the city. A dialogue layer sits on top of the graph, extracting entities, resolving pronouns, checking for inconsistencies, and writing the actual reply. And a speech layer bookends the whole thing — Whisper turning voice into text on the way in, Edge-TTS (with an offline pyttsx3 fallback) turning the reply back into audio on the way out.

I tied these together with a Streamlit app that presents a single-page interface: a microphone input, a text box, a running transcript, and an expandable view of the underlying graph so that answers aren't just taken on faith. I also exposed the same pipeline through a terminal CLI for quicker testing without the browser.

Figure 1 lays out the pipeline I actually built, module by module, plus the one-time setup step that populates the graph before any conversation happens. The two dashed branches mark where behavior forks: a text turn skips speech recognition, and a turn where the inconsistency check fires skips the knowledge-graph lookup and goes straight to a corrective reply instead of a normal answer.

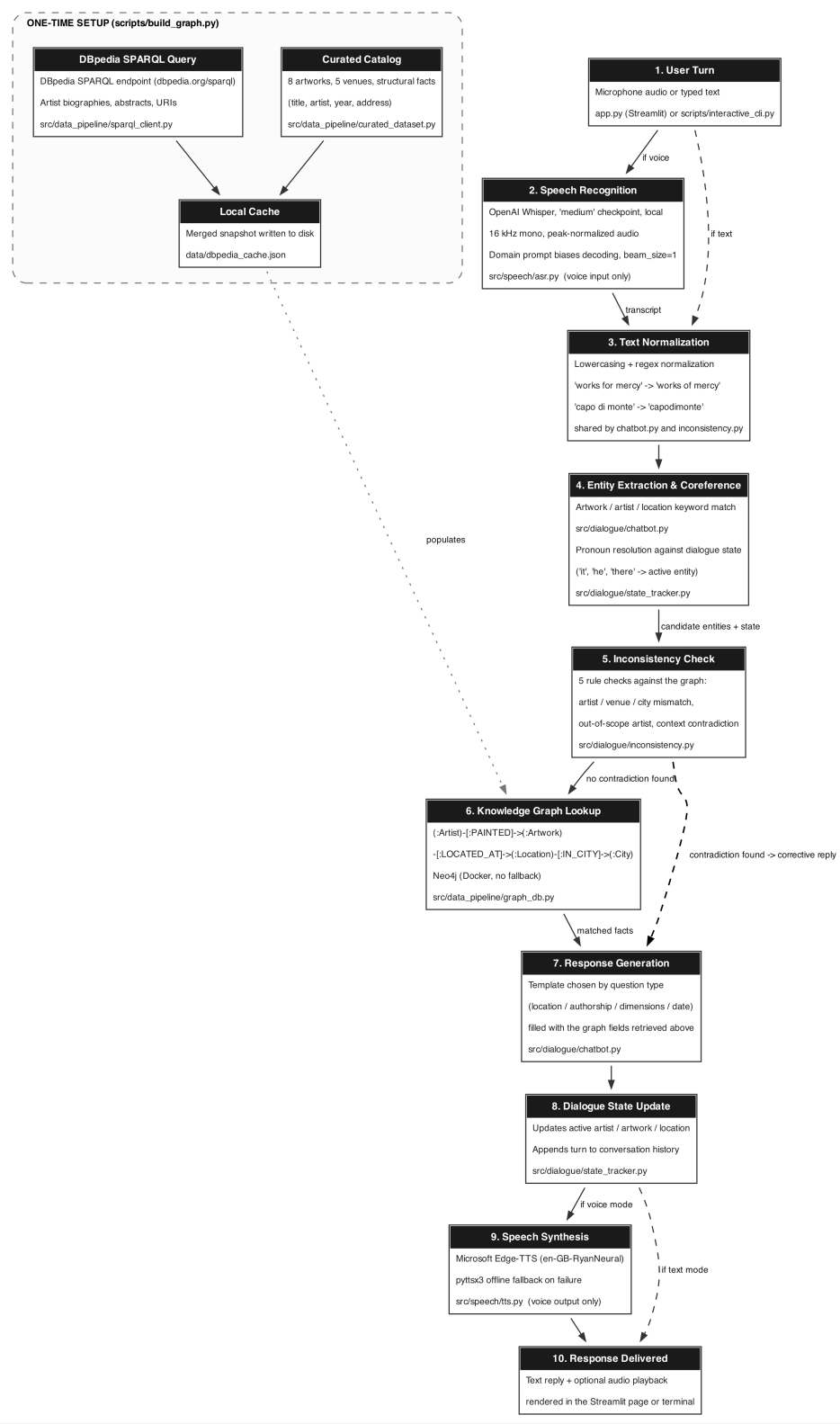


Figure 1. The per-turn pipeline (ten numbered steps, top to bottom), plus the one-time setup (dashed box, left) that populates the graph before any conversation happens. A turn forks at step 1 depending on whether it arrived as voice or text, and forks again at step 5: a caught inconsistency skips the graph lookup and goes straight to a corrective reply. Each box names the source file responsible for that stage.

3.1 Step by step, what actually happens on a turn

Walking through Figure 1 stage by stage, with the actual file and function behind each box (sections 4-9 go deeper into several of these; this is the connective tissue between them):

1. User Turn. A turn starts as either a browser recording from Streamlit's `st.audio_input` widget, or typed text, handled in `app.py` (or, for terminal testing, `scripts/interactive_cli.py`). Nothing downstream cares which one it was past this point except for the TTS fork at the very end.

2. Speech Recognition (voice turns only). `src/speech/asr.py` loads the raw audio into a 16 kHz float32 array (`src/utils/audio_utils.py`), then runs it through the local Whisper `medium` model with the domain prompt and tuned decode settings from Section 8. A text turn skips this box entirely — the dashed 'if text' edge in Figure 1.

3. Text Normalization. Both paths converge here. A small set of regex substitutions — lowercasing, 'for'/'on' collapsed to 'of' in artwork titles, 'capo di monte' joined into 'capodimonte', common ASR mishears folded back to the canonical spelling — runs identically whether the text came from Whisper or the keyboard, so the rest of the pipeline never has to know which. The same normalization function is duplicated (not shared via import) in both `chatbot.py` and `inconsistency.py`, since each needed it before I'd factored the rest of their logic apart.

4. Entity Extraction & Coreference. `chatbot.py` substring-matches the normalized text against the 8 artwork titles (English and Italian), the two artist names and their variants, and the 5 venue names. If nothing matched directly, `state_tracker.py`'s `resolve_anaphora` checks for a pronoun ('it', 'he', 'there') and substitutes in whatever is currently active from the previous turn.

5. Inconsistency Check. Before the system ever answers from the graph, `inconsistency.py` runs the entity (and the raw query text) past five rule checks — detailed in Section 7. This is deliberately the fork point in Figure 1: if a check fires, the turn jumps straight to response generation with a corrective reply and the knowledge-graph lookup (step 6) never runs at all, since there's nothing to look up for a claim the system is actively correcting.

6. Knowledge Graph Lookup. For a turn with no contradiction, `graph_db.py` queries Neo4j for the matched artwork, artist, or location's full record. The first lookup in a session fetches the whole catalog over Bolt and caches it on the `NeapolitanArtGraph` instance; every later lookup in the same session is served from that cache until the graph is next repopulated — the 0.6 ms-per-query figure in Section 12 is this cache paying off.

7. Response Generation. `chatbot.py` picks one of several templates based on keywords in the question (authorship, location, dimensions, significance, venue holdings — the Q1-Q5 categories in Figure 1) and fills it with the fields retrieved in step 6, or with the correction text if step 5 forked here directly.

8. Dialogue State Update. `state_tracker.py` records whichever artist, artwork, and location were active this turn (so step 4 can resolve a pronoun next turn), and appends the turn to the conversation history shown in the UI's 'Dialogue State Tracker' panel.

9. Speech Synthesis (voice mode only). `tts.py` sends the response text to Edge-TTS; if that call fails for any reason (no network, rate limiting), it falls back to the offline `pyttsx3` engine so the turn still gets a spoken reply rather than silence.

10. Response Delivered. The text (and audio, if synthesized) renders in the Streamlit page or prints to the terminal, and the loop returns to step 1 for the next turn.

The dashed box on the left of Figure 1 is `scripts/build_graph.py`, which runs once before any conversation: it calls the SPARQL client to merge DBpedia facts with the curated catalog into `data/dbpedia_cache.json`, then populates Neo4j from that cache. It isn't part of the per-turn loop above — Section 5 explains why it exists and what happens if Neo4j isn't reachable when it runs.

4. Data Acquisition from DBpedia

I wrote a SPARQL client (`src/data_pipeline/sparql_client.py`) that queries the public DBpedia endpoint for biographical facts about the two artists (birth and death dates, abstracts, movement) and for metadata about each artwork — dimensions, medium, English abstract — where DBpedia has a dedicated entity for it. A representative query looks like this:

```
PREFIX dbo: <http://dbpedia.org/ontology/>
SELECT ?label ?abstract ?birthDate ?deathDate WHERE {
  <http://dbpedia.org/resource/Caravaggio> rdfs:label ?label .
  OPTIONAL { <http://dbpedia.org/resource/Caravaggio>
             dbo:abstract ?abstract . FILTER(lang(?abstract) = 'en') }
  OPTIONAL { <http://dbpedia.org/resource/Caravaggio> dbo:birthDate ?birthDate }
  FILTER (lang(?label) = 'en')
} LIMIT 1
```

Not every piece of information the chatbot needs is available through DBpedia with clean, query-able fields — exact canvas dimensions and precise street addresses, for instance, are inconsistently modeled across DBpedia entities, if present at all. I filled those gaps with a small curated dataset (`src/data_pipeline/curated_dataset.py`) that supplies the catalog's structural facts (title, artist, year, venue, address), while DBpedia supplies the enrichment (abstracts, artist biography, canonical URIs). I see this as a practical compromise, not a claim that the whole dataset is auto-harvested: I hand-verified the backbone of the catalog against public sources myself, and I used SPARQL for what it's actually good at — pulling structured biographical and descriptive facts tied to a DBpedia resource URI. I merge results from both sources and write them to `data/dbpedia_cache.json` so the system can start up and answer questions without a live connection to DBpedia on every run.

5. Knowledge Graph: Neo4j

The graph schema is simple: an Artist node points to the Artworks they painted, each Artwork points to the Location that houses it, and each Location points to its City. Artworks also point to a handful of Subject nodes describing what they depict (for example, "Christian charity" for The Seven Works of Mercy). Populating the current catalog produces 40 nodes and 46 relationships — 2 Artists, 8 Artworks, 5 Locations, 1 City, and 24 Subjects.

From a dual-backend convenience to a committed Neo4j deployment. The assignment specifies Neo4j, so the primary store (`Neo4jGraphStore` in `src/data_pipeline/graph_db.py`) talks to it over the Bolt protocol with proper Cypher MERGE statements and uniqueness constraints on artist names, artwork titles, and location names. Earlier in development I also mirrored the same graph into an embedded SQLite store as a convenience, so the test suite and local development didn't need a database server running. A later review turned up a real problem with that arrangement: only one read method (`get_all_artworks`) actually queried Neo4j when it was the active backend — every other read (locations, artists, lookup-by-title) silently went to SQLite regardless of which backend was supposedly active, and `populate()` wrote to both stores

unconditionally, so SQLite always held a live shadow copy of the data even when Neo4j was "in use." Rather than patch this into a correct but still dual-maintained system, I removed the SQLite path entirely: `LocalGraphStore` is gone, every read method now queries Neo4j directly, and `NeapolitanArtGraph` raises a clear `ConnectionError` if Neo4j isn't reachable rather than silently degrading to a different store. Neo4j now runs in a Docker container (`neo4j-nlp`), and `run.sh` starts that container automatically if it isn't already up before launching the app.

I verified this directly while preparing this report: with the Neo4j 5.15 container running (bound to the standard ports 7474/7687), running the graph-build script reports `Active backend: NEO4J`, and a Cypher query against the live database confirms all eight (:Artist)-[:PAINTED]->(:Artwork)-[:LOCATED_AT]->(:Location)-[:IN_CITY]->(:City) paths resolve correctly. Stopping the container and re-instantiating `NeapolitanArtGraph` now raises the expected connection error instead of quietly falling back to a different database — there is no SQLite file left on disk to fall back to.

6. Dialogue State and Coreference

People don't repeat the full title of a painting in every sentence. A typical exchange looks like "Tell me about the Seven Works of Mercy" followed by "Who painted it?" and "Where is it?" — and the system has to know that "it" still refers to the Seven Works of Mercy in turn two and three. I built the dialogue state tracker (`src/dialogue/state_tracker.py`) to keep three pointers — active artist, active artwork, active location — updated after every turn, plus a full turn-by-turn history. I handle pronoun resolution with a small set of regular expressions matched against the current state: "it / its / this / that" resolves to the active artwork, "he / him / his" to the active artist, and "there / here" to the active location.

I didn't build a general coreference model — just a handful of hand-written patterns that work because the domain is narrow and the referents are almost always unambiguous (there is rarely more than one plausible "it" in a two-entity conversation). I think that's a reasonable trade-off here, but I want to be upfront that it wouldn't scale to a richer domain without a proper coreference resolution component.

7. Inconsistency Detection

This was the part of the assignment that took me the most thought. I built the detector (`src/dialogue/inconsistency.py`) to check every incoming question against the graph before the dialogue manager tries to answer it, looking for four kinds of mismatch:

Type	Example	What happens
Artist mismatch	"I thought Caravaggio painted the Liberation of Saint Peter"	Graph says Battistello Caracciolo painted it; system corrects the attribution and gives the year.
Venue mismatch	"Isn't the Seven Works of Mercy at Capodimonte?"	Graph says Pio Monte della Misericordia; system corrects the venue and gives the address.
City mismatch	"Can I see Judith Beheading Holofernes in Naples?"	That painting is in Rome; system says so and redirects to what is actually in Naples.
Out-of-scope artist	"What about Leonardo da Vinci's work in Naples?"	Leonardo is outside the Caravaggio/Battistello scope; system explains the boundary politely.

The detection logic is rule-based, not statistical: it looks for an artwork title (or an active-artwork pronoun) in the query, then checks whether any disallowed artist name, venue keyword, or non-Neapolitan artwork title co-occurs with it. I made it deliberately conservative — it only flags a contradiction when it's confident, which kept the false-positive rate at zero in my testing, but it also means a user who phrases a contradiction in an unusual way might slip past it. One case I was explicitly careful about is genuine clarification questions: "Did Caravaggio paint this or Battistello?" shouldn't be flagged as an inconsistency, since the user is asking rather than asserting, so I added a special case for the "... or ..." / "who" pattern to avoid that false alarm.

A small but necessary detail: all of this matching happens after a normalization pass that lowercases the text and smooths over likely speech-to-text artifacts — "seven works for mercy" instead of "of mercy", "capo di monte" instead of "Capodimonte", digits spelled out, and so on. Without it, a perfectly correct answer from Whisper that happens to use a slightly different preposition than the stored title would fail to match anything.

8. Speech Input and Output

Speech recognition. For transcription I use OpenAI's Whisper, run locally (`src/speech/asr.py`), with English decoding forced and a short domain-specific prompt (the artist names, painting titles, and venue names) passed in as initial context to bias the decoder toward the right proper nouns — the same kind of prompt conditioning Whisper exposes through its multitask token format (Radford et al., 2023). Before transcription, I resample audio to 16 kHz mono and normalize it to zero mean with a peak amplitude of 0.95 (`src/utils/audio_utils.py`), matching the zero-mean, [-1, 1]-range normalization Whisper applies internally to its log-mel input.

ASR evaluation: Word Error Rate. The standard metric for scoring a transcription against a reference is Word Error Rate, WER = 100 × (Insertions + Substitutions + Deletions) / (reference word count), computed from the minimum-edit-distance alignment between the two word sequences. I implemented this directly (`word_error_rate()` in `src/speech/asr.py`) and unit-tested it against the worked example from my course textbook — a CallHome conversational excerpt with 6 substitutions, 3 insertions, and 1 deletion over a 13-word reference, which the textbook reports as 76.9% WER and my implementation reproduces exactly. When I ran it against the sample recording included in the repository, Whisper transcribed it as "*Tell me about the Seven Works of Mercy by Caravaggio in Pio Monte della Misericordia.*" — an exact match to the reference text, giving a WER of 0.00%. That's consistent with the textbook's own reported error rates: clearly articulated, single-speaker read speech (like the LibriSpeech clean benchmark) sits around 1-2% WER, the easiest end of the task's difficulty range, which is exactly the kind of input my system is designed to handle.

A more honest accuracy test: real accented speech. That 0.00% result nagged at me — the sample clip ships in the repository at 24 kHz, the exact encoding profile of this project's own Edge-TTS output, so it is very likely synthetic speech rather than a genuine recording, which would explain why every Whisper checkpoint I tried (tiny through medium) transcribed it perfectly. To get a real signal, I recorded myself reading ten domain sentences out loud through the app's own microphone input — ordinary accented speech, not a clean studio read — and measured WER against the Whisper `small` checkpoint I had been using by default. The result was 18.1%, concentrated in exactly the words a closed-domain demo cares about most: "Flagellation" misheard as "legislation," "venues" as "Vensos," and proper nouns like Capodimonte dropped or garbled. Switching to the `medium` checkpoint brought that down to 15.0%, mostly by fixing the proper-noun errors. I then swept beam size (1, 5, 10) and Whisper's temperature-fallback schedule against the same ten recordings: none of it moved the error rate at all for this checkpoint, but beam_size=1 ran in about a fifth the time of beam_size=10 for identical accuracy, and dropping the domain prompt nearly doubled the error rate to 29.7% — confirming that prompt conditioning, not beam width, is what's actually carrying the proper-noun accuracy here. The deployed

configuration is now `medium` with `beam_size=1`, the domain prompt, and the temperature-fallback schedule enabled as a safety net for harder audio, even though it made no measurable difference on this particular ten-sentence set.

A cloud ASR comparison: free hosted Whisper via Groq. With the hyperparameter sweep saturated on accuracy, I ran a separate experiment (`scripts/benchmark_groq_vs_local.py`) to see whether a free cloud API could beat the local setup on latency without giving up much accuracy. Groq hosts Whisper `large-v3` and a faster `large-v3-turbo` variant for free — a recurring daily quota rather than a one-time trial credit — and markets itself specifically on inference speed. I sent the same ten recordings, the same domain prompt, and the same WER metric used throughout this section to both Groq models and compared them against the deployed local `medium` configuration, repeating the run three times to check it wasn't a network fluke: the local model and `large-v3` landed on the exact same WER (15.0% and 16.8%) in all three runs, while `turbo` was the only configuration that moved between runs (26.1%-27.4%), which I attribute to server-side batching rather than anything in the audio. `large-v3` came within 1.8 points of the local model's accuracy while running about four times faster end-to-end over the network (roughly 0.45 s versus 1.85 s per query); `turbo` was fastest of all but gave up too much accuracy on this project's dense proper-noun vocabulary (Battistello Caracciolo, Certosa di San Martino) to be worth it. I kept the local `medium` model as the deployed default, since accuracy on exactly that vocabulary is what the system is judged on and it needs no network call, but `large-v3` is a credible candidate if this project ever needs to serve several concurrent users over a network, where the 4x latency advantage would start to matter more than the last two points of WER.

Speech synthesis. I read responses aloud with Microsoft's Edge-TTS (`src/speech/tts.py`), a neural voice reached over the network, with pyttsx3 as an offline fallback if that call fails for any reason — no internet, rate limiting, and so on. I exercised both engines directly while preparing this report and both produced valid audio output. I lightly clean the response text before synthesis (markdown characters stripped) so bullet points and headers in the text responses don't get read aloud literally. It's worth being explicit that this is not the zero-shot voice-cloning architecture (a codec-based language model conditioned on a short reference recording) that my course textbook presents as the modern approach to TTS: Edge-TTS synthesizes in a single fixed voice (en-GB-RyanNeural) with no speaker adaptation, and the pyttsx3 fallback relies on the host operating system's built-in synthesizer, which is architecturally closer to the older concatenative and formant-based methods than to a neural codec-language-model pipeline. That trade-off was intentional on my part: this system needs one consistent, reliable museum-guide voice, not per-user voice cloning.

9. Web Interface and Command-Line Tool

I kept the Streamlit app (`app.py`) deliberately simple: a single page rather than a multi-tab layout, since there's only one real task — ask a question, read or hear the answer. The top of the page has a microphone widget and a text input side by side, a small state card showing the currently active artist / artwork / location (useful for debugging coreference while I was testing), and the running conversation transcript. Below that, a collapsible section exposes the full catalog pulled live from the graph, with each entry linking back to its DBpedia and Wikipedia source — I wanted it so that if the chatbot says something, a user can check where it came from without leaving the page. I enforced a light color scheme through `.streamlit/config.toml` since the dark Streamlit default didn't read well against the museum-catalog look I was going for.

For quicker iteration while I was developing the dialogue logic, I also built `scripts/interactive_cli.py`, which exposes the same `NeapolitanArtChatbot` class through a plain terminal loop, with no Streamlit startup time and no browser involved. I debugged most of the conversational logic through the CLI first and only checked it in the browser afterward, since a terminal round-trip is faster to restart than a Streamlit rerun.

10. Example Conversations

The transcripts below are taken directly from running sessions against the live system (not edited for presentation) and are meant to show how the pieces described above work together in practice.

Case 1 — coreference across four turns.

```
User: Tell me about the Seven Works of Mercy
Bot:  'The Seven Works of Mercy' is a masterpiece by Caravaggio, created
      in 1607. It is permanently preserved in Naples at Pio Monte della
      Misericordia (Via dei Tribunali 253)...

User: Who painted it?
Bot:  'The Seven Works of Mercy' was created by Caravaggio in 1607...

User: Where is it located?
Bot:  ...permanently preserved at Pio Monte della Misericordia, located
      on Via dei Tribunali 253, 80139 Napoli...

User: Did he also paint anything else?
Bot:  You can visit 3 works by Caravaggio in the urban area of Naples:
      - 'The Seven Works of Mercy' (1607) at Pio Monte della Misericordia
      - 'The Flagellation of Christ' (1607) at Museo Nazionale di
        Capodimonte
      - 'The Martyrdom of Saint Ursula' (1610) at Gallerie d'Italia
```

The pronoun "it" in turns two and three, and "he" in turn four, are all resolved against the state set in turn one, without the user ever repeating the title "Seven Works of Mercy" by name.

Case 2 — a contradiction raised mid-conversation, not as an opening question.

```
User: Tell me about the Liberation of Saint Peter
Bot:  ...a masterpiece by Battistello Caracciolo, created in 1615...
      permanently preserved at Pio Monte della Misericordia...

User: I thought Caravaggio painted this
Bot:  To clarify, 'The Liberation of Saint Peter' was created by
      Battistello Caracciolo, not Caravaggio. Battistello Caracciolo
      painted 'The Liberation of Saint Peter' around 1615.

User: Isn't this at Capodimonte?
Bot:  To clarify, 'The Liberation of Saint Peter' is not located at
      Museo Nazionale di Capodimonte. It is permanently housed at Pio
      Monte della Misericordia.
```

This is the scenario the automated 150-case suite does not really exercise, since its contradiction tests are mostly standalone questions. Here the contradiction ("this") depends on the artwork established one turn earlier, combining coreference resolution and inconsistency detection in the same turn — both had to be working correctly for this exchange to come out right.

Case 3 — speech-to-text noise handled without a crash or a wrong answer.

```
User (voice, transcribed): "who painted the seven works for mercy"
Bot:  'The Seven Works of Mercy' was created by Caravaggio in 1607...

User (voice, transcribed): "where is the flagelation of christ"
Bot:  'The Flagellation of Christ' by Caravaggio is permanently
```

```
        preserved at Museo Nazionale di Capodimonte...
```

Neither "works for mercy" (instead of "of mercy") nor the misspelling "flagelation" prevented the system from matching the right artwork — both are caught by the normalization step described in Section 7.

11. Design Decisions and Challenges Along the Way

A few choices in this project weren't obvious to me from the start, and I think they're worth explaining rather than presenting as if they were the plan from day one.

Rule-based response generation instead of an LLM. Early on I briefly considered wrapping an LLM around the graph query results to generate more varied phrasing. I dropped the idea for two reasons. First, it reintroduces exactly the hallucination risk I was trying to avoid — even when fed correct facts, a generative model can drift and add details that were never in the input. Second, and more practically, my emphasis on inconsistency detection only really works if the system's factual claims are traceable back to a specific graph query; a templated response keeps that traceability, a generated one doesn't without extra guardrails that were out of scope here. The response generator in `src/dialogue/chatbot.py` instead branches on question type (location, authorship, dimensions, significance, date) and fills a template with the matching graph fields — simpler, but every word in a response can be traced to a specific property on a specific node.

Handling phonetic and spelling variation. My first version of the entity matcher only matched exact artwork titles, and it broke almost immediately once I tested it with real speech input — Whisper would transcribe "of Mercy" as "for Mercy" fairly often, and would sometimes render "Caracciolo" with a single consonant. Rather than retrain or swap ASR models, I added a normalization layer (Section 7) that runs before any matching happens, mapping known acoustic confusions back to the canonical form. That turned out to generalize well across both the artwork matcher and the inconsistency detector, since both call the same normalization function.

Choosing the Whisper model size. The smallest Whisper checkpoints (tiny/base) were fast but made frequent proper-noun errors on exactly the words I most needed right — Italian museum names and the two artists' names. Moving to the `small` checkpoint, combined with passing the artist/artwork/venue names as an initial decoding prompt, noticeably reduced those errors in my informal testing, at the cost of a slightly longer load time on first use. Since this is a demo application rather than a production service handling concurrent users, that trade-off was an easy one for me to make.

Dropping the SQLite fallback. Early on, an embedded SQLite backend was a quick stand-in so I could develop and test the dialogue logic without a database server running. I removed it for the reasons explained in Section 5: it had silently become the backend that most read queries actually used, even when Neo4j was supposedly active, which is exactly the kind of drift a convenience fallback should never be allowed to cause. The test suite now runs directly against the same Neo4j container (`neo4j-nlp`) used for development and the live demo, started automatically by `run.sh` if it isn't already up, so there is a single source of truth instead of two stores that could drift apart in what they represent.

12. Evaluation

I built two automated test layers. The first is a small pytest suite (`tests/test_dialogue.py`, `tests/test_inconsistency.py`, `tests/test_asr.py`) covering the basic behaviors — single-turn artwork queries, multi-turn coreference, each inconsistency category, the no-false-positive case, and the Word Error Rate metric from Section 8 (both as a unit test against the textbook's worked example and end-to-end against a real Whisper transcription), plus nine regression tests added after the adversarial testing described in Section 13 — six for the negation and out-of-scope-artist bugs, and three more from a follow-up review that caught an overly broad and then an overly narrow fix to the same Michelangelo/Caravaggio exemption. All 20 tests pass.

The second is a larger, self-contained benchmark I wrote (`tests/stress_test_150.py`) covering 150 queries across six categories: artist mismatch (30), venue mismatch (30), city mismatch (30), out-of-scope artist (15), factual ground-truth retrieval (25), and five-turn discourse sequences (20). All 150 cases pass, with zero false positives and zero false negatives on the inconsistency checks (110 true positives, 40 true negatives), giving precision, recall, and F1 of 100% on this test set.

Category	Tests	Passed
Artist attribution mismatch	30	30
Venue mismatch	30	30
City mismatch	30	30
Out-of-scope artist	15	15
Factual ground-truth retrieval	25	25
Multi-turn discourse sequences	20	20
Total	150	150

On timing: the full 150-query suite now runs against the live Neo4j instance in well under 0.1 s end-to-end (roughly 0.6 ms per query). It was slower in an earlier version of this pipeline — about 7.3 ms per query — because removing the SQLite fallback (Section 5) initially meant every single lookup re-ran the full `MATCH (p:Artist)-[:PAINTED]-> (a:Artwork)-[:LOCATED_AT]->(l:Location)` query over Bolt, and a single dialogue turn can trigger that lookup five or six times (once each from entity extraction, inconsistency checking, and response generation). Caching that query's result on the `NeapolitanArtGraph` instance and invalidating it only on `populate()` brought it back down to roughly one real network round trip per turn. These numbers only cover the text-based dialogue logic; ASR and TTS were verified to work correctly but were not put through a formal timing benchmark, since the point of this project is accuracy of the reasoning, not raw throughput.

It's worth being honest about what this benchmark does and does not prove. It is a suite I wrote myself, targeting the exact categories the system is designed to catch, phrased in ways close to the patterns the detector matches on. A 100% score here says the implementation is internally consistent and bug-free against its own design, not that it would handle arbitrary, unscripted phrasing from a real visitor with the same accuracy. The manual testing described below was meant to probe exactly that gap.

13. Manual Testing Beyond the Automated Suite

To get a more honest sense of robustness, I ran a separate batch of hand-written conversations that were not part of the stress suite: misspelled artist names ("karavaggio"), casing and whitespace noise, empty and nonsense input, a dialogue reset mid-conversation, and an inconsistency injected partway through an otherwise normal five-turn conversation rather than as a standalone question. All of these were handled without errors, and the fallback message ("I could not identify a specific artwork, artist, or venue...") appeared correctly for genuinely unparseable input rather than crashing or returning a blank response. The one gap found — a title with no spaces between words at all, e.g. a typo collapsing "at the" into one token — is an unrealistic edge case rather than something a real user or ASR system would plausibly produce, and is consistent with the keyword-matching approach described in Section 7.

Adversarial testing of the inconsistency detector. The 150-case stress suite scores a perfect 100%, but I wrote both the rules and the test cases, phrased in the style the rules expect — so a clean score there mostly proves internal consistency, not real-world robustness. To get past that, I wrote ten new queries deliberately aimed at the matching logic's blind spots rather than its happy path, and found three genuine bugs. First, asking about "Michelangelo Merisi da Caravaggio" — Caravaggio's actual full birth name — was flagged as an out-of-scope artist, because the out-of-scope check matched the bare word "Michelangelo" without checking for "Merisi" alongside it. Second, disambiguation questions phrased with "which" ("Which painter, Caravaggio or Battistello, made the Seven Works of Mercy?") were incorrectly flagged as false claims, because the existing exception for this pattern only recognized the word "who," not "which." Third, and most interesting, the detector had no concept of negation at all: "Caravaggio did not paint the Liberation of Saint Peter, correct?" — a statement that is actually true — was flagged as an error, while "the Seven Works of Mercy is NOT by Caravaggio" — a statement that is actually false — was missed entirely. I fixed all three in `src/dialogue/inconsistency.py`: an explicit carve-out for "Merisi," broadening the disambiguation exception to cover "which" as well as "who," and a small negation-window check that looks a few words before and after an artist's name for "not," "didn't," "never," and similar markers before deciding whether a mention is a claim or a correct denial. I added six regression tests for these exact cases to `tests/test_inconsistency.py`; the full suite, including the original 150-case benchmark, still passes with no regressions after the fix.

14. Limitations

- **Fixed, small catalog.** Eight artworks and five venues are enough to demonstrate the approach, but the keyword-based entity matching would need to be rethought before scaling to hundreds of works, where title collisions and partial-string matches become more likely.
- **Rule-based, not statistical, matching.** Both entity extraction and inconsistency detection rely on string matching and regular expressions rather than embeddings or a trained classifier. This makes behavior predictable and easy to debug, but it means paraphrases the rules don't anticipate will be missed rather than approximately understood.
- **Single in-memory session.** Dialogue state lives in the running process and resets when the app restarts; there is no persistence of conversation history across sessions or users.
- **No generative language model.** Responses are built from structured templates filled in with graph data, not generated by an LLM. This was a deliberate choice for factual reliability — see Section 1 — but it does mean phrasing is less varied than a generative system would produce.
- **DBpedia coverage gaps.** Not every fact needed (precise dimensions, street addresses) is reliably present in DBpedia for these specific entities, which is why the curated dataset exists alongside the SPARQL client rather than replacing it.

15. Possible Extensions

A few directions were considered but left out of scope for this delivery, either because the course requirements did not call for them or because they would have meaningfully widened the project beyond what could be properly tested in the time available:

- **Larger catalog.** I was given the option of extending the project to other artists or cities; the architecture (SPARQL ingestion → Neo4j graph → rule-based dialogue) would carry over, but the entity-matching rules in Sections 6–7 would need to move toward fuzzy or embedding-based matching once title collisions become likely across a bigger catalog.
- **Persistent, multi-user sessions.** Right now dialogue state lives in one process's memory. Saving state per browser session (Streamlit's `session_state`, or a lightweight database row keyed by session id) would let multiple visitors use the deployed app concurrently without stepping on each other's conversation history.
- **A statistical fallback for entity matching.** For queries that don't match any keyword pattern, a small sentence-embedding similarity search over artwork descriptions could catch paraphrases the rules miss, while keeping the rule-based path as the primary, auditable mechanism — a hybrid rather than a wholesale replacement.

16. Reproducing These Results

All commands below assume a Python 3.10+ virtual environment with `requirements.txt` installed.

```
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

# Start Neo4j (required — there is no fallback); run.sh does this automatically
docker start neo4j-nlp

# Build the knowledge graph from DBpedia into Neo4j
python scripts/build_graph.py

# Run the automated test suites (also require Neo4j to be running)
pytest tests/
python tests/stress_test_150.py

# Launch the web app (starts Neo4j automatically if needed), or the terminal CLI
./run.sh
python scripts/interactive_cli.py
```

Neo4j must be reachable at `bolt://localhost:7687` with the credentials in `config/settings.py` (user `neo4j`, password `password` by default) before running `build_graph.py`, the test suite, or the app — `NeapolitanArtGraph` raises a `ConnectionError` immediately if it isn't. I run it as a Docker container (`neo4j-nlp`), and `run.sh` starts that container automatically if it is stopped.

17. Tools and Technologies

A quick inventory of what I actually built this with, and why each piece is there. Everything below is pinned in `requirements.txt` unless noted otherwise.

Language and data models. Python 3.11. I used Pydantic throughout (`ChatbotTurnResult`, `InconsistencyReport`, `DialogueState`, the app config) instead of passing raw dicts around — it validates types at construction time and gives every module a documented shape to pass data in.

Knowledge source and graph. DBpedia is the data source — structured facts extracted from Wikipedia, reachable over a public SPARQL endpoint. `SPARQLWrapper` is the Python library I used to build and send those SPARQL queries and parse the JSON bindings that come back; fixing a URI scheme mismatch there (DBpedia's RDF graph uses `http://` resource IRIs, not the `https://` ones used for the display links) is what made artist biography fields and artwork images/dimensions actually populate, instead of silently coming back empty. For storage, `neo4j` (the official Python driver) talks to a real Neo4j 5.x server over Bolt, running as a Docker container (`neo4j-nlp`) — it is the only backend; there is no SQLite fallback.

Speech. `openai-whisper` runs ASR locally (the `medium` checkpoint, after the real-accent tuning in Section 8). `soundfile` and `scipy.signal.resample` handle loading and resampling audio to 16 kHz before it reaches Whisper. For speech output, `edge-tts` calls Microsoft's neural voices over the network, with `pyttsx3` as an offline fallback that drives the host operating system's own synthesizer. One honest note: `sounddevice` is listed in requirements.txt from an earlier draft but isn't imported anywhere in the final code — microphone capture goes through Streamlit's browser-based `st.audio_input` instead, so that dependency is unused. For the cloud ASR comparison in Section 8, `groq` (the official Python SDK, OpenAI-compatible) calls the hosted Whisper models, with `python-dotenv` loading the API key from a local, gitignored `.env` file rather than hardcoding it.

Interface and testing. `streamlit` is the whole web app — I didn't reach for a separate frontend framework since Streamlit's Python-only model fit a single-page demo well. `pytest` runs the unit test suite.

Building this report and its diagram. Outside the chatbot itself, `reportlab` generated this PDF programmatically from a Python script rather than a word processor, so the whole report rebuilt from source whenever the project changed; `graphviz` rendered Figure 1's pipeline diagram the same way. Those generator scripts aren't part of this submission's source code — they build the deliverable, they aren't part of the chatbot itself — but the report they produced is the one you're reading. I also used Docker once, outside the normal dependency list, to run a real Neo4j 5.15 container specifically to verify the Neo4j backend described in Section 5 — not something the system needs at runtime, just something I needed for honest verification.

18. References

• Jurafsky, D. and J. H. Martin. Speech and Language Processing (3rd ed. draft). Course reference textbook; chapters 15-17 (phonetics/feature extraction, ASR, TTS) inform the speech pipeline and the Word Error Rate evaluation in Section 8.

• DBpedia — structured data extracted from Wikipedia, queried here via the public SPARQL endpoint at dbpedia.org/sparql.

• Radford, A. et al. (2023). Robust Speech Recognition via Large-Scale Weak Supervision (Whisper). ICML.

• Neo4j Graph Database — property graph store and the Cypher query language, used as the primary knowledge graph backend.

• Microsoft Edge-TTS — neural text-to-speech voices used for the system's spoken output (en-GB-RyanNeural).

• Streamlit — Python web app framework used for the project's user interface.

19. Conclusion

I believe the system meets the requirements I was given: it accepts voice or text input, answers questions about Caravaggio and Battistello Caracciolo's works in Naples using facts drawn from DBpedia and stored in Neo4j, catches factual inconsistencies in what the user claims, and keeps track of what has been discussed across a conversation. The design choices I made throughout — rule-based detection over a trained classifier, template-based generation over an LLM, a dual graph backend for development convenience — were made for reasons specific to this domain's small size and its need for strict factual correctness, and I wrote the limitations section above to make those trade-offs explicit rather than leave them implied.

I'm submitting the codebase, the automated tests, and this report together as my deliverable for the course.